

Taller 2: Identidad y la puerta de entrada

Plataforma de monitoreo de dispositivos médicos

Trabajo en grupo · en clase

La pregunta. Ya tienen varios servicios corriendo. ¿Cómo hace un servicio para confiar en la identidad de un usuario **sin preguntarle a otro servicio en cada petición**, y sin que esa confianza dependa de una cabecera sin verificar?

Esta guía describe **qué** debe lograr su sistema al terminar, no un procedimiento de copiar y pegar. Las decisiones de arquitectura de la Sección 2 están fijas y no se discuten en clase; **cómo las implementan —en qué lenguaje, con qué librerías, con o sin ayuda de IA— es libertad de cada grupo**, siempre que el resultado cumpla los requisitos y el grupo pueda explicarlo.

Regla. Pueden usar IA para generar código. No pueden entregar código que el grupo no sepa explicar línea por línea si se les pregunta.

1 Arquitectura de destino

```
navegador --> gateway :8080 --+--> auth-service --> auth-db
                               +--> backend      --> backend-db
```

La **puerta** (gateway) es la recepción de un edificio: indica el piso y exige el carnet, pero no sabe si el carnet es falso. Cada **servicio** vuelve a mirarlo en su propia puerta. **Auth** es la única oficina que emite carnets.

1.1 El recorrido

1. El cliente manda correo y contraseña a `/api/auth/login`. Es el único momento en que una contraseña viaja.
2. Auth verifica contra el hash y entrega dos credenciales: **access** (JWT, corto) y **refresh** (largo, solo para renovar).
3. En cada petición siguiente el cliente manda `Authorization: Bearer <access>`.
4. El backend comprueba **por su cuenta** firma, exp, iss y aud con la clave pública de auth. No le pregunta a auth.
5. Si el access venció, el cliente llama a `/api/auth/refresh`. Auth invalida el usado y entrega un par nuevo.

2 Decisiones fijas

No se discuten en clase. Cómo las implementan es libre; que se cumplan, no.

- **Firma asimétrica, no secreto compartido.** Auth firma con una clave privada que ningún otro servicio tiene. Los demás comprueban con la pública. Un `JWT_SECRET` compartido no cumple.
- **La puerta no comprueba firmas.** Exige que la cabecera exista; la firma y los claims los comprueba cada servicio. Rodear la puerta no debe bastar.

- **Access corto** (recomendado 15 min) y **refresh rotado** (recomendado 8 h), guardado como **hash** en la base de auth. Usar un refresh lo invalida.
- **401 vs 403**. Sin token, firma inválida o vencido → 401. Identidad válida sin permiso suficiente → 403.
- **Un contenedor Docker por servicio**, con su propia imagen. No un contenedor con dos procesos adentro.

3 Requisitos

REQUISITO 1 Separar servicios y bases

auth-service y backend son contenedores distintos, cada uno con su propia base. auth-db guarda solo usuarios y refresh tokens; backend-db conserva los datos del Taller 1. Ningún servicio se conecta a la base del otro.

Recomendación. Redes Docker separadas, y un usuario de base por servicio — así el aislamiento lo impone la infraestructura, no la disciplina.

Comprobación. Backend no puede leer la tabla de usuarios de auth, ni por permisos ni por red.

REQUISITO 2 Una sola puerta

El frontend habla con una sola dirección. La puerta enruta por prefijo (/api/auth/* a auth, el resto a backend) y resuelve CORS en un solo lugar.

Recomendación. nginx como proxy reverso es la opción más simple, pero cualquier gateway equivalente sirve. Dejen los puertos internos accesibles dentro de la red de contenedores: los necesitan para el Requisito 4.

Comprobación. El frontend conoce una URL. Si detienen auth, login falla con error de la puerta (502), no con error de conexión del navegador.

REQUISITO 3 Identidad con firma asimétrica y JWKS

Auth firma con una clave privada que solo él tiene. Cualquier otro servicio puede comprobar esa firma consultando un endpoint público, sin conocer la privada.

Cómo lograr GET /api/auth/jwks:

- Generen un par RSA de 2048 bits. La privada se monta **solo** en auth — nunca en el repositorio ni en el entorno de otro servicio.
- Devuelvan la clave pública en formato **JWKS** (un JSON con un arreglo keys). Es el formato que las librerías de JWT ya saben consumir; no inventen uno propio.
- Incluyan un kid en el token y en cada entrada del JWKS, para poder rotar claves sin invalidar los tokens vigentes.
- En backend, usen una librería de JWKS de su ecosistema. No implementen la comprobación de firma a mano.
- **Cacheen el JWKS en memoria** tras la primera consulta. Pedirlo en cada petición reintroduce la dependencia de red que este patrón evita.

Si usan IA. Pídanle explícitamente que compruebe iss, aud y exp además de la firma. Es común que un primer intento generado solo compruebe la firma.

Comprobación. Un token de auth lo acepta backend. La clave privada no aparece fuera de auth. Agregar una segunda clave al JWKS no invalida los tokens firmados con la primera.

REQUISITO 4 Defensa en profundidad

Si alguien llama al puerto interno de backend, saltándose la puerta, la petición sigue exigiendo un token válido.

Recomendación. El middleware de comprobación vive **dentro** de backend. No deleguen en una cabecera que la puerta agregue (X-User-Id o similar) y que backend acepte sin cuestionar.

Comprobación.

```
curl backend:4000/<ruta protegida>
# -> 401

curl backend:4000/<ruta protegida> -H "Authorization: Bearer <token>"
# -> igual que pasando por la puerta
```

4 Endpoints estandarizados

Obligatorios en forma, para que cualquiera pueda probar el sistema de cualquier grupo. La implementación interna es libre.

Ruta	Método	Contrato
/api/auth/register	POST	{email, password, nombre} → 201, o 409 si el correo existe
/api/auth/login	POST	{email, password} → {accessToken, refreshToken, expiresIn}, o 401
/api/auth/refresh	POST	{refreshToken} → par nuevo; el anterior deja de servir
/api/auth/jwks	GET	Público, sin token. Devuelve el JSON Web Key Set vigente

Cabecera siempre Authorization: Bearer <token>. Las cuatro rutas de arriba son públicas; todo lo demás exige token.

5 Material de referencia

Punto de partida, no lectura obligatoria. Si van a usar IA para generar código, estas fuentes les sirven para **comprobar** que lo generado sigue el estándar.

5.1 JWKS y JWT

Fuente	Para qué
RFC 7517	El estándar JWKS. Vayan a la Sección 4 para ver qué campos lleva cada clave (kid, kty, use, alg).

RFC 7519	El estándar JWT. La Sección 4.1 define los claims registrados: <code>iss</code> , <code>sub</code> , <code>aud</code> , <code>exp</code> .
jwt.io	Explicación visual, y un depurador donde pueden pegar un token y ver qué lleva adentro. Útil para depurar.
El JWKS de Google	Un JWKS real , en vivo. Ábranlo en el navegador: así debe verse la respuesta de su endpoint.
RFC 8725	Buenas prácticas de JWT. Qué errores de seguridad son comunes y cómo evitarlos.

5.2 Librerías para comprobar firmas

No implementen la comprobación a mano. Según su stack:

- Node: github.com/panva/jose (moderna, incluye cliente JWKS remoto) o [auth0/node-jwks-rsa](#) (solo el cliente JWKS, con caché incluido).
- Python: [PyJWT](#) — vean la sección *Retrieve RSA signing keys from a JWKS endpoint*.
- Go: [golang-jwt/jwt](#).

5.3 nginx como puerta de entrada

Fuente	Para qué
Guía de proxy reverso	El concepto y la configuración mínima. Empiecen por aquí.
Módulo proxy	Referencia de <code>proxy_pass</code> y las cabeceras que conviene reenviar.
CORS en MDN	Qué cabeceras hacen falta y cómo funciona la petición <i>preflight</i> .
Imagen oficial de Docker	Dónde montar su archivo de configuración dentro del contenedor.

Ojo con `proxy_pass`. Que la ruta termine o no en `/` cambia el resultado: con `/` nginx quita el prefijo de la ruta, sin `/` lo conserva. Es la causa más común de un 404 que parece inexplicable.

6 Entregable

1. El sistema corriendo en Docker, un contenedor por servicio, con los cuatro requisitos comprobables en vivo.
2. Cómo verificaron cada requisito. No basta con decir que funciona.
3. Herramientas usadas, incluida la IA: para qué parte, y qué tuvieron que corregir de lo generado.

7 Para el Taller 3

Hoy resolvieron **autenticación**: cualquier servicio sabe quién es el usuario. Falta **autorización de recurso**: decidir si ese usuario puede tocar un objeto específico, cuando el dato que dice de quién es vive en otro servicio.